

ESP32forth et l'interface WiFi

Marc PETREMANN



Version 1.0 - 17. mai 2026

Table des matières

Préambule.....	3
Extension du vocabulaire WiFi.....	3
ESP32 et son interface WiFi.....	5
Étapes d'une connexion complète.....	6
Initialisation du mode Station.....	6
Le vocabulaire WiFi.....	7
WIFI_MODE_AP (-- 2).....	7
WIFI_MODE_APSTA (-- 3).....	8
WIFI_MODE_NULL (-- 0).....	9
WIFI_MODE_STA.....	10
WiFi.begin (ssid-z password-z --).....	10
WiFi.config (ip dns gateway subnet --).....	10
WiFi.disconnect (--).....	11
WiFi.enableIPv6.....	12
WiFi.localIP (-- ip).....	12
WiFi.macAddress (addr --).....	12
WiFi.mode (n --).....	13
WiFi.getTxPower (-- powerx4).....	14
WiFi.setTxPower (powerx4 --).....	15
WiFi.scanNetworks (-- n).....	16
WiFi.SSID (id -- zstr).....	17
WiFi.RSSI (id -- rssi).....	17
WiFi.channel (id -- chan).....	17
WiFi.encryptionType (id -- enc).....	17
WiFi.scanDelete.....	17
WiFi.softAP (ssid/0 password/0 -- success).....	17
WiFi.softAPBroadcastIP (-- n).....	18
WiFi.softAPConfig (localip gateway subnet -- success).....	19
WiFi.softAPdisconnect (fl – success).....	20
WiFi.softAPIP.....	21
WiFi.softAPNetworkID.....	21
WiFi.softAPgetStationNum.....	21
WiFi.status.....	21
WiFi.linkLocalIPv6.....	21
WiFi.globalIPv6.....	21
Index lexical.....	23

Préambule

Ce manuel est destiné à la prise en main de l'interface WiFi pour ESP32forth

Extension du vocabulaire WiFi

Certains mots ont été rajoutés au vocabulaire WiFi. Voici la version étendue de ce vocabulaire à prendre en compte dans le fichier source ESP32forth.ino :

```
# define OPTIONAL_WIFI_SUPPORT \
/* WiFi */ \
XV(WiFi, "WiFi.config", WIFI_CONFIG, \
    WiFi.config(ToIP(n3), ToIP(n2), ToIP(n1), ToIP(n0)); DROPn(4)) \
XV(WiFi, "WiFi.begin", WIFI_BEGIN, WiFi.begin(c1, c0); DROPn(2)) \
XV(WiFi, "WiFi.disconnect", WIFI_DISCONNECT, WiFi.disconnect(true)) \
XV(WiFi, "WiFi.status", WIFI_STATUS, PUSH WiFi.status()) \
XV(WiFi, "WiFi.macAddress", WIFI_MAC_ADDRESS, WiFi.macAddress(b0); DROP) \
XV(WiFi, "WiFi.localIP", WIFI_LOCAL_IPS, PUSH FromIP(WiFi.localIP())) \
XV(WiFi, "WiFi.mode", WIFI_MODE, WiFi.mode((wifi_mode_t) n0); DROP) \
XV(WiFi, "WiFi.setTxPower", WIFI_SET_TX_POWER, \
WiFi.setTxPower((wifi_power_t) n0); DROP) \
XV(WiFi, "WiFi.getTxPower", WIFI_GET_TX_POWER, PUSH WiFi.getTxPower()) \
XV(WiFi, "WiFi.softAP", WIFI_SOFTAP, n0 = WiFi.softAP(c1, c0); NIP) \
XV(WiFi, "WiFi.softAPIP", WIFI_SOFTAP_IP, PUSH FromIP(WiFi.softAPIP())) \
XV(WiFi, "WiFi.softAPBroadcastIP", WIFI_SOFTAP_BROADCASTIP, PUSH \
FromIP(WiFi.softAPBroadcastIP())) \
XV(WiFi, "WiFi.softAPNetworkID", WIFI_SOFTAP_NETWORKID, PUSH \
FromIP(WiFi.softAPNetworkID())) \
XV(WiFi, "WiFi.softAPConfig", WIFI_SOFTAP_CONFIG, n0 = \
WiFi.softAPConfig(ToIP(n2), ToIP(n1), ToIP(n0))) \
XV(WiFi, "WiFi.softAPdisconnect", WIFI_SOFTAP_DISCONNECT, n0 = \
WiFi.softAPdisconnect(n0)) \
XV(WiFi, "WiFi.softAPgetStationNum", WIFI_SOFTAP_GET_STATION_NUM, PUSH \
WiFi.softAPgetStationNum()) \
/* WiFi scan */ \
/* Récupère le nombre de réseaux WiFi détectés */ \
XV(WiFi, "WiFi.scanNetworks", WIFI_SCANNETWORKS, PUSH \
(cell_t)WiFi.scanNetworks()) \
/* Récupère le SSID du réseau indexé par n0 */ \
XV(WiFi, "WiFi.SSID", WIFI_SSID, { String s = WiFi.SSID(n0); n0 = (cell_t) \
strdup(s.c_str()); }) \
/* Récupère la puissance (RSSI) du réseau indexé par n0 */ \
XV(WiFi, "WiFi.RSSI", WIFI_RSSI, n0 = \
(cell_t)WiFi.RSSI(n0)) \
/* Récupère le canal (channel) du réseau indexé par n0 */ \
XV(WiFi, "WiFi.channel", WIFI_CHANNEL, n0 = \
(cell_t)WiFi.channel(n0)) \
/* Récupère le type de cryptage du réseau indexé par n0 */ \
```

```
XV(WiFi, "WiFi.encryptedType", WIFI_ENCRYPTIONTYPE, n0 =  
(cell_t)WiFi.encryptedType(n0)) \  
/* Efface les résultats du scan de la mémoire */ \  
XV(WiFi, "WiFi.scanDelete", WIFI_SCANDELETE, WiFi.scanDelete())
```

ESP32 et son interface WiFi

L'interface WiFi de l'ESP32 est un sous-système puissant qui permet au microcontrôleur de communiquer sans fil via la pile TCP/IP. Sous **ESP32forth**, cette interface est pilotée par des mots qui font le pont avec les bibliothèques C++ natives. Elle fonctionne principalement selon trois modes : **Station (STA)** pour se connecter à un routeur, **Access Point (AP)** pour créer son propre réseau, ou le mode hybride **AP+STA**.

Le processus suit une logique séquentielle : on définit d'abord le mode avec **WiFi.mode**, puis on initie la connexion ou le service avec **WiFi.begin** ou **WiFi.softAP**. Comme la connexion est **asynchrone**, le programme doit surveiller l'état via **WiFi.status** avant d'échanger des données. Une fois connecté, l'ESP32 obtient une adresse IP (récupérable avec **WiFi.localIP**) et peut ouvrir des **sockets** pour agir comme un client ou un serveur Web, permettant ainsi de piloter du matériel à distance via un simple navigateur ou des requêtes HTTP.

Le WiFi 4 (Standard 2,4 GHz) est le standard pour la grande majorité des projets.

- **ESP32 Original (Série WROOM/WROVER)** : Le plus courant. WiFi + Bluetooth Classic + BLE.
- **ESP32-S2** : WiFi uniquement (pas de Bluetooth). Très économe en énergie.
- **ESP32-S3** : WiFi + Bluetooth LE. Très puissant, idéal pour l'IA et les écrans.
- **ESP32-C3** : WiFi + Bluetooth LE. Modèle économique à un seul cœur (architecture RISC-V).

Les modèles "Next Gen" (WiFi 6), sont plus récents (2024-2026) et offrent une meilleure gestion des réseaux encombrés :

- **ESP32-C6** : WiFi 6 + Bluetooth 5 + Zigbee/Thread (pour la domotique "Matter").
- **ESP32-C5** : C'est le premier modèle **Dual-Band** (2,4 GHz et **5 GHz**).

L'exception (Sans WiFi), une série récente fait exception :

- **ESP32-P4** : Cette carte est ultra-puissante (400 MHz) et possède des capacités vidéo, mais elle n'a **aucune connectivité sans fil** intégrée. Elle est destinée aux applications industrielles ou aux interfaces locales.

Comment reconnaître l'interface WiFi sur votre carte ?

Regardez le module métallique (souvent argenté) sur votre carte : si vous voyez une ligne en forme de "S" ou de "zig-zag" tracée sur le circuit imprimé ou une petite pièce en céramique à une extrémité, c'est l'antenne WiFi.



Le protocole **ESP-NOW** permet de communiquer entre cartes ESP32 par WiFi sans nécessiter la connection à un routeur.

Étapes d'une connexion complète

Lancer **WiFi.begin** ne suffit pas à garantir que vous êtes connecté. Le processus est asynchrone. Voici comment structurer une connexion propre :

Initialisation du mode Station

Avant de se connecter, il est crucial de dire à l'ESP32 de se comporter comme un client (Station) et non comme un point d'accès.

```
WiFi.disconnect \ Optionnel : nettoie les anciennes sessions  
1 WiFi.mode    \ 1 = Mode Station (STA)
```

Une fois **WiFi.begin** lancé, vous devez surveiller le statut.

```
z" SSID_RESEAU" z" PASSWORD" WiFi.begin  
  
: wait-wifi ( -- )  
  begin  
    WiFi.status 3 <> \ 3 correspond à WL_CONNECTED  
  while  
    ." En attente du WiFi..." cr  
    1000 ms  
  repeat  
    ." Connecté ! IP : " WiFi.localIP .ip cr ;
```

Le vocabulaire WiFi

Le vocabulaire **wifi** est accessible en exécutant **wifi vlist** :

```
WIFI_MODE_APSTA WIFI_MODE_AP WIFI_MODE_STA WIFI_MODE_NULL WiFi.config
WiFi.begin
WiFi.disconnect WiFi.status WiFi.macAddress WiFi.localIP WiFi.mode
WiFi.setTxPower WiFi.getTxPower WiFi.softAP WiFi.softAPIP
WiFi.softAPBroadcastIP WiFi.softAPNetworkID WiFi.softAPConfig
WiFi.softAPdisconnect WiFi.softAPgetStationNum WiFi.scanNetworks
WiFi.SSID WiFi.RSSI WiFi.channel WiFi.encryptionType WiFi.scanDelete
```

WIFI_MODE_AP (-- 2)

Constante. Valeur 2.

Dans l'univers de l'ESP32 et de Forth, **WIFI_MODE_AP** (Access Point) est l'opposé du mode Station. Au lieu de se connecter à une box internet, l'ESP32 devient lui-même la "box".

Il crée son propre réseau WiFi auquel vous pouvez connecter votre smartphone ou votre ordinateur :

- **Configuration initiale** : Permet de configurer la carte ESP32 sans connaître les identifiants WiFi du lieu.
- **Contrôle direct** : Idéal pour piloter un robot ou un objet connecté en plein milieu d'un champ, sans routeur à proximité.
- **Interface locale** : Héberger une page Web de contrôle accessible via une IP fixe.

Utilisation :

```
WIFI_MODE_AP WiFi.mode      \ Définit le mode sur WIFI_MODE_AP
z" Mon_ESP32_Reseau"        \ SSID (Nom du réseau)
z" motdepasse123"           \ Mot de passe (min 8 caractères)
WiFi.softAP                  \ Notez l'utilisation de softAP au lieu de begin
```

Pour un point d'accès, on utilise souvent **WiFi.softAP** plutôt que **WiFi.begin**. La commande **WiFi.begin** cherche à rejoindre un réseau, tandis que **WiFi.softAP** le crée.

Lorsque l'ESP32 est en mode AP, il possède des caractéristiques par défaut :

- **Adresse IP par défaut** : C'est presque toujours **192.168.4.1**.
- **Nombre de clients** : Généralement limité à 4 ou 8 appareils simultanés sur un ESP32.
- **Canal (Channel)** : Par défaut sur le canal 1 (peut être modifié pour éviter les interférences).

WIFI_MODE_APSTA (-- 3)

Constante. Valeur 3.

Le mode `WIFI_MODE_APSTA` (souvent associé à la valeur **3**) est le mode le plus polyvalent de l'ESP32. Il permet à la puce de fonctionner simultanément comme une **Station (STA)** et comme un **Point d'Accès (AP)**.

C'est comme si votre ESP32 avait deux cartes réseau virtuelles partageant la même antenne physique.

Ce mode est la base de deux fonctionnalités célèbres dans le monde de l'IoT :

- **Le Portail Captif de Configuration** : Votre ESP32 essaie de se connecter à votre box (STA). S'il échoue, il active son propre réseau (AP) pour que vous puissiez vous y connecter avec votre téléphone et lui donner les nouveaux identifiants via une page web, sans jamais perdre le contact.
- **Le Répéteur WiFi (Range Extender)** : L'ESP32 reçoit le signal internet d'un côté et le rediffuse de l'autre pour étendre la portée du réseau.
- **Passerelle (Gateway)** : Il peut collecter des données de capteurs connectés à son propre réseau AP et les envoyer vers un serveur distant via sa connexion STA.

Ce mode est la base de deux fonctionnalités célèbres dans le monde de l'IoT :

- **Le Portail Captif de Configuration** : Votre ESP32 essaie de se connecter à votre box (STA). S'il échoue, il active son propre réseau (AP) pour que vous puissiez vous y connecter avec votre téléphone et lui donner les nouveaux identifiants via une page web, sans jamais perdre le contact.
- **Le Répéteur WiFi (Range Extender)** : L'ESP32 reçoit le signal internet d'un côté et le rediffuse de l'autre pour étendre la portée du réseau.
- **Passerelle (Gateway)** : Il peut collecter des données de capteurs connectés à son propre réseau AP et les envoyer vers un serveur distant via sa connexion STA.

Pour l'activer, on empile la valeur **3** avant d'appeler `WiFi.mode` :

```
WIFI_MODE_APSTA WiFi.mode          \ Active le mode hybride AP+STA

\ Configuration de la partie AP (son propre réseau)
z" Mon_Reseau_Local" z" mdp12345" WiFi.softAP

\ Configuration de la partie STA (se connecte à la box)
z" Ma_Box_Internet" z" mdp_secret" WiFi.begin
```

Bien que très puissant, ce mode a quelques limites à connaître :

- **Le Canal (Channel)** : Comme il n'y a qu'une seule antenne, le mode AP et le mode STA doivent impérativement utiliser le **même canal WiFi**. Si votre box

change de canal, l'ESP32 devra ajuster son propre réseau AP pour suivre, ce qui peut déconnecter brièvement les appareils qui y sont reliés.

- **Consommation** : C'est le mode qui consomme le plus d'énergie puisque la radio est sollicitée pour maintenir deux types de connexions.
- **Adressage** : L'ESP32 aura deux adresses IP différentes : une sur le réseau de la box (via **WiFi.localIP**) et une pour son propre réseau (via **WiFi.softAPIP**).

Dans ce mode, vous pouvez interroger les deux identités :

```
WiFi.localIP .ip    \ L'adresse donnée par votre box (ex: 192.168.1.45)
WiFi.softAPIP .ip   \ L'adresse de sa propre bulle WiFi (ex: 192.168.4.1)
```

C'est le mode "couteau suisse" par excellence pour rendre vos projets robustes et faciles à configurer !

WIFI_MODE_NULL (-- 0)

Constante. Valeur 0.

Le mode **WIFI_MODE_NULL** est, comme son nom l'indique, l'état "zéro" ou "vide" de l'interface WiFi de l'ESP32.

C'est le mode par défaut au démarrage (avant toute configuration) ou un mode de désactivation. Lorsqu'il est actif :

- L'interface WiFi n'est ni en mode Station (STA), ni en mode Point d'Accès (AP).
- La pile TCP/IP est virtuellement arrêtée pour la partie sans fil.
- L'ESP32 ne cherche pas à se connecter et n'émet aucun signal.

Il y a deux raisons principales pour appeler ce mode :

- **Économie d'énergie** : Le WiFi est le composant le plus gourmand de l'ESP32. Passer en **WIFI_MODE_NULL** (souvent via la commande **0 WiFi.mode**) permet de couper la radio et de réduire drastiquement la consommation électrique si votre projet doit effectuer des calculs locaux sans communiquer.
- **Réinitialisation propre** : Avant de changer radicalement de configuration WiFi (passer d'un mode AP complexe à un mode STA), certains développeurs forcent le mode NULL pour s'assurer que les ressources mémoires du WiFi sont libérées.

```
WIFI_MODE_NULL WiFi.mode \ Désactive l'interface WiFi
```

Ne confondez pas **WIFI_MODE_NULL** avec le mode **Power Save**. Le mode NULL coupe l'interface, tandis que le Power Save maintient la connexion avec le routeur tout en s'endormant entre deux balises (beacons).

C'est le mode "silence radio" total. Utile si votre ESP32 fonctionne sur batterie et n'a besoin d'envoyer des données qu'une fois par heure, par exemple.

WIFI_MODE_STA

Constante. Valeur 1.

L'ESP32 dispose de plusieurs modes Wi-Fi. Le mode Station est le mode le plus courant :

- **Le concept** : Dans ce mode, l'ESP32 se comporte comme votre smartphone ou votre ordinateur. Il **se connecte à un réseau Wi-Fi existant** (comme celui de votre box internet ou un partage de connexion).
- **L'objectif** : Cela lui permet d'accéder à Internet, d'envoyer des données vers un serveur cloud, ou de communiquer avec d'autres appareils sur le même réseau local.

Le mode **WIFI_MODE_AP** (Access Point) transforme la carte ESP32 en sa propre borne Wi-Fi sur laquelle d'autres appareils viennent se connecter.

```
\ 1. Initialiser le Wi-Fi
login-wifi

\ 2. Définir le mode sur STATION (en utilisant la constante)
WIFI_MODE_STA wifi-start

\ 3. Se connecter au point d'accès avec le SSID et le mot de passe
z" Mon_Nom_De_Box" z" Mon_Mot_De_Passe" wifi-connect

\ 4. (Optionnel) Vérifier si l'IP est attribuée
.wifi
```

WiFi.begin (ssid-z password-z --)

la commande **WiFi.begin** est le point d'entrée pour connecter la carte ESP32 à un réseau local. FORTH traite cette commande de manière interactive, ce qui permet de tester sa connectivité en temps réel via le terminal.

Le mot **WiFi.begin** attend deux chaînes de caractères (Z-strings) sur la pile : le **nom du réseau (SSID)** et le **mot de passe** :

```
z" Mon_SSID" z" Mon_Mot_De_Passe" WiFi.begin
```

Le SSID est le nom du routeur tel qu'il est diffusé sur votre réseau WiFi local. Le mot de passe est celui associé à ce SSID.

WiFi.config (ip dns gateway subnet --)

Dans l'environnement ESP32forth, **WiFi.config** est la commande qui vous permet de reprendre le contrôle sur l'adressage réseau. Par défaut, l'ESP32 utilise le **DHCP** (il demande une adresse IP automatiquement à votre box), mais **WiFi.config** permet de lui assigner une **IP statique** (fixe).

C'est essentiel si vous créez un serveur web sur votre ESP32, car cela évite que l'adresse IP ne change à chaque redémarrage.

La commande **WiFi.config** attend généralement plusieurs adresses IP (sous forme d'entiers 32 bits) sur la pile.

WiFi.disconnect (--)

Le mot **WiFi.disconnect** est l'interrupteur logiciel qui permet de rompre proprement la liaison sans fil de l'ESP32. Bien qu'elle paraisse simple, elle joue plusieurs rôles cruciaux dans la gestion de la stabilité et de la consommation de votre module.

Lorsqu'elle est appelée, l'ESP32 envoie un paquet de "désassociation" au routeur (ou au client s'il est en mode AP).

- Elle stoppe les tentatives de reconnexion automatique.
- Elle libère une partie des ressources mémoire allouées à la session active.
- Elle fait basculer le **WiFi.status** vers la valeur **6** (**WL_DISCONNECTED**).

Il y a trois scénarios principaux :

- **Changement de réseau** : Avant de lancer un nouveau **WiFi.begin** sur un autre SSID, il est fortement recommandé de faire un **WiFi.disconnect** pour éviter que l'ancienne session ne crée des conflits.
- **Économie d'énergie** : Si votre projet n'envoie des données qu'une fois par heure, faire un **WiFi.disconnect** permet de couper la radio et d'économiser la batterie entre deux envois.
- **Réinitialisation (Reset)** : Si la connexion est instable ou "gelée", un appel à **WiFi.disconnect** suivi d'un délai (**1000 ms**) et d'un nouveau **WiFi.begin** est la méthode standard pour forcer une reconnexion propre.

En Forth, la commande est simple et ne prend pas de paramètres sur la pile :

```
WiFi.disconnect
```

Avant de commencer une nouvelle configuration, voici une bonne pratique :

```
: wifi-reset ( -- )  
  WiFi.disconnect  
  500 ms  
  0 WiFi.mode      \ Passe en mode NULL pour tout effacer  
  500 ms  
  ." WiFi réinitialisé." cr  
  ;
```

WiFi.enableIPv6

Non installé – prévu dans une version future

WiFi.localIP (-- ip)

La commande **WiFi.localIP** est un mot de type "getter" : elle ne prend rien sur la pile, mais elle y dépose l'adresse réseau actuellement attribuée à votre ESP32 par le routeur.

Lorsque vous appelez **WiFi.localIP**, ESP32forth interroge sa pile TCP/IP interne (LwIP).

- **Entrée** : Rien (--).
- **Sortie** : Un entier de 32 bits représentant l'IP (**ip**).

L'adresse est stockée sous forme de **4 octets packés** dans un seul nombre. Par exemple, si votre IP est **192.168.1.15**, le nombre sur la pile ne sera pas "192", mais une valeur brute comme **251766976** (en décimal) ou **\$0F01A8C0** (en hexadécimal).

WiFi.macAddress (addr --)

Le mot **WiFi.macAddress** permet de récupérer l'identifiant physique unique de votre puce ESP32. Contrairement à l'adresse IP (qui peut changer selon le réseau), l'adresse MAC est "gravée" en usine dans le matériel.

MAC signifie *Media Access Control*. C'est une suite de **6 octets** (souvent affichés en hexadécimal) qui sert de numéro de série mondial unique pour chaque interface réseau.

- **Format type** : **24:0A:C4:XX:XX:XX**
- Les trois premiers octets identifient le constructeur (Espressif Systems dans notre cas).

Exemple d'utilisation :

```
\ store current mac Address
create myMac 6 allot

\ get mac address for current ESP32 card and store result in myMac
: getMyMac ( -- )
  myMac WiFi.macAddress
;

\ display MAC address in hex format
: .mac { mac-addr -- }
  base @ hex
  6 0 do
    mac-addr i + c@ <# # # #> type
  i 5 < if
```

```

        [char] : emit
    then
    loop
    base !
;

```

Utilité :

1. **Identification unique** : Si vous avez 10 capteurs ESP32 sur un réseau, l'adresse MAC est le meilleur moyen de savoir "qui est qui" côté serveur.
2. **Sécurité (Filtrage MAC)** : Si votre routeur est configuré pour n'accepter que certains appareils, vous devez lui donner cette adresse.
3. **Génération d'ID** : On l'utilise souvent pour créer un nom de réseau WiFi (SSID) unique pour chaque appareil, par exemple **MonRobot-240AC4**.

La carte ESP32 possède en réalité **deux** adresses MAC distinctes (souvent décalées d'une unité) :

- Celle de l'interface **Station** (utilisée avec **WiFi.macAddress**).
- Celle de l'interface **SoftAP** (utilisée avec **WiFi.softAPmacAddress**).

WiFi.mode (n --)

Le mot **WiFi.mode** est le sélecteur de configuration matérielle de votre ESP32. Avant de tenter de vous connecter ou de créer un réseau, vous devez impérativement indiquer à la puce quel "rôle" elle doit jouer.

C'est une étape cruciale car elle initialise les piles logicielles et active la radio de manière appropriée.

Le mot attend un entier sur la pile qui correspond au mode souhaité,

Voici les quatre modes principaux que vous rencontrerez :

Valeur	Constante C++	Description
0	WIFI_MODE_NULL	Mode Off . Désactive le WiFi pour économiser l'énergie.
1	WIFI_MODE_STA	Station . L'ESP32 se connecte à un routeur (comme un smartphone).
2	WIFI_MODE_AP	Access Point . L'ESP32 crée son propre WiFi pour que d'autres s'y connectent.
3	WIFI_MODE_APSTA	Hybride . L'ESP32 fait les deux en même temps.

Si vous essayez de faire un **WiFi.begin** sans avoir fait **1 WiFi.mode**, la connexion risque d'échouer ou d'être instable car les ressources "Station" n'auront pas été allouées par le système d'exploitation (FreeRTOS) sous-jacent. Exemple :

```
: init-sta ( -- )
  WIFI_MODE_STA WiFi.mode \ On passe en mode Station
  z" Mon_SSID"
  z" Mon_MDP"
  WiFi.begin \ On lance la connexion
  ." Mode Station activé" cr ;
```

Le mode **WIFI_MODE_APSTA** est très puissant. Il permet par exemple à l'ESP32 de rester connecté à votre box internet tout en proposant un petit portail de secours si vous perdez la connexion, ou d'agir comme un répéteur WiFi.

Si vous changez de mode en cours d'exécution, il est souvent préférable de faire un petit **WiFi.disconnect** ou d'attendre quelques millisecondes pour laisser le temps à la puce de réinitialiser la transmission radio.

WiFi.getTxPower (-- powerx4)

Le mot **WiFi.getTxPower** permet de mesurer la puissance en émission radio de votre ESP32. Il retourne la puissance d'émission actuelle de l'antenne.

La puissance de transmission (TX Power) détermine la portée de votre signal WiFi.

- Elle s'exprime en **dBm** (décibel-milliwatts).
- Sur un ESP32, cette valeur se situe généralement entre **8 dBm** (puissance faible) et **20 dBm** (puissance maximale).

Le mot ne prend rien sur la pile et retourne un entier représentant la puissance.

```
WiFi.getTxPower .
```

1. **Diagnostic de portée** : Si vous n'arrivez pas à capter votre ESP32 à plus de 2 mètres, vérifiez avec **getTxPower** si la radio n'est pas bridée par logiciel.
2. **Gestion de l'énergie** : Plus la puissance est élevée, plus l'ESP32 consomme de courant (ampères) et plus il chauffe.
3. **Conformité** : Dans certains pays ou environnements sensibles, il peut être nécessaire de limiter la puissance pour respecter les normes locales ou ne pas brouiller d'autres appareils.

Il existe souvent le mot jumeau **WiFi.setTxPower** qui permet de modifier cette valeur. Par exemple, pour réduire la consommation :

```
: low-power-wifi ( -- )
  40 WiFi.setTxPower \ Définit une puissance plus faible
  ." Puissance réglée à : " WiFi.getTxPower . cr ;
```

La puissance maximale réelle dépend aussi de la qualité de l'alimentation électrique de votre carte. Si votre alimentation est faible, monter la puissance TX peut provoquer un crash de l'ESP32 lors de l'émission des données.

WiFi.setTxPower (powerx4 --)

En **ESP32forth** (comme dans l'environnement Arduino pour ESP32), la fonction **WiFi.setTxPower** permet de **régler la puissance d'émission (Tx Power) du signal Wi-Fi**.

En termes simples, cela sert à ajuster l'intensité avec laquelle l'ESP32 "parle" (émet ses ondes) pour communiquer avec la box internet ou d'autres appareils.

Par défaut, l'ESP32 émet à sa puissance maximale pour garantir la meilleure portée possible. Cependant, modifier ce paramètre est très utile dans deux cas précis :

- **Économie d'énergie (Batterie) :** Le Wi-Fi est l'un des composants les plus gourmands de l'ESP32. Si votre projet fonctionne sur batterie et que l'ESP32 est proche de la box, baisser la puissance d'émission permet de réduire drastiquement la consommation électrique.
- **Réduction des interférences :** Si vous avez des dizaines d'ESP32 dans une même pièce, baisser leur puissance évite qu'ils ne se parasitent les uns les autres.

La puissance s'exprime en **dBm** (décibel-milliwatt). Pour l'ESP32, la plage de valeurs acceptée va généralement de **8 dBm** (puissance minimale) à **20 dBm** (puissance maximale par défaut).

Voici un aperçu de l'impact des valeurs :

- **20 dBm (Puissance max) :** Portée maximale, mais consommation électrique élevée (courant pouvant dépasser 240 mA lors des transmissions).
- **14 dBm (Puissance moyenne) :** Bon compromis pour un usage domestique standard.
- **8 dBm (Puissance min) :** Idéal si l'ESP32 est à quelques mètres seulement du routeur ; économise beaucoup d'énergie.

Voici comment cela se traduit en Forth :

```
\ 1. Configurer la puissance au maximum (20 dBm)
20 WiFi.setTxPower

\ 2. Ou configurer la puissance au minimum pour économiser la batterie (8 dBm)
8 WiFi.setTxPower
```

WiFi.scanNetworks (-- n)

Le mot **WiFi.scanNetworks** permet à l'ESP32 de **scanner l'environnement** à la recherche des réseaux Wi-Fi disponibles. C'est exactement ce que fait votre smartphone lorsque vous ouvrez les paramètres Wi-Fi et que la liste des box internet des voisins apparaît.

Le scan de réseaux est très utile pour plusieurs scénarios :

- **Sélection automatique** : Permettre à l'ESP32 de chercher s'il capte le Wi-Fi de votre maison *ou* celui de votre partage de connexion mobile, et de se connecter automatiquement au signal le plus fort.
- **Interface de configuration** : Si vous créez un portail de configuration (en mode **softAP**), vous pouvez afficher à l'utilisateur la liste des réseaux détectés pour qu'il n'ait plus qu'à cliquer sur le sien et entrer son mot de passe.
- **Analyse d'environnement** : Mesurer la qualité du signal (RSSI) pour optimiser le placement de votre ESP32 par rapport à votre routeur.

Le mot **WiFi.scanNetworks** renvoie un **nombre entier** sur la pile. Ce nombre correspond au **nombre de réseaux Wi-Fi trouvés** (par exemple **5** si elle a détecté 5 box internet). Si le scan échoue ou est encore en cours, il peut retourner un code d'erreur (comme **-1** ou **-2**).

Voici comment on l'utilise généralement pour obtenir le nombre de réseaux :

```
\ 1. Le Wi-Fi doit être initialisé en mode Station au préalable
WIFI_MODE_STA wifi-start

\ 2. Lancer le scan
WiFi.scanNetworks ( -- n )

\ 3. Afficher le nombre de réseaux trouvés
cr ." Réseaux trouvés : " .
```

La commande **WiFi.scanNetworks** ne fait *que* lancer la recherche et retourner le nombre de réseaux. Pour afficher concrètement le nom (SSID) ou la puissance de chaque réseau, ESP32forth fournit généralement d'autres mots complémentaires qui s'utilisent dans une boucle.

Par exemple, si **WiFi.scanNetworks** a mis le chiffre **5** sur la pile, on utilise des fonctions compagnes comme :

- **WiFi.SSID** : pour obtenir le nom du réseau à un index donné.
- **WiFi.RSSI** : pour obtenir la puissance du signal en dBm (plus le chiffre est proche de 0, meilleur est le signal, ex: -50 dBm est excellent, -90 dBm est très faible).

WiFi.SSID (id -- zstr)

WiFi.RSSI (id -- rssi)

WiFi.channel (id -- chan)

WiFi.encryptionType (id -- enc)

WiFi.scanDelete

WiFi.softAP (ssid/0 password/0 -- success)

Le mot **WiFi.softAP** permet de configurer l'ESP32 en mode **Point d'Accès Logiciel** (Software Access Point).

Contrairement au mode Station (**WIFI_MODE_STA**) où l'ESP32 se connecte à une box existante, **WiFi.softAP** transforme l'ESP32 en **propre borne Wi-Fi**. Il génère son propre réseau sans fil auquel d'autres appareils (smartphone, ordinateur) peuvent se connecter directement.

Ce mode est particulièrement utile dans plusieurs scénarios :

- **Configuration initiale (Portail captif) :** Quand vous vendez ou donnez un objet connecté, il ne connaît pas le Wi-Fi de l'utilisateur. L'ESP32 crée son propre Wi-Fi. L'utilisateur s'y connecte avec son smartphone pour configurer le vrai nom et mot de passe de sa box internet.
- **Contrôle direct (Hors ligne) :** Si vous fabriquez une voiture télécommandée ou un système domotique en pleine forêt, sans aucune box internet aux alentours, votre smartphone peut se connecter directement à l'ESP32 pour le piloter.

Quand l'ESP32 active le **softAP**, il crée un réseau local et démarre un serveur DHCP invisible.

- **Son adresse IP par défaut** est généralement **192.168.4.1**.

- Tout appareil qui s'y connecte recevra une adresse du type **192.168.4.2**, **192.168.4.3**, etc.

Pour démarrer ce mode, la fonction a besoin au minimum de deux paramètres : le **nom du réseau (SSID)** que vous voulez créer et son **mot de passe** (qui doit faire au moins 8 caractères).

En Forth, on passe les arguments sur la pile sous forme de "Z-strings" (chaînes terminées par un zéro binaire) avant d'appeler la fonction :

```
\ 1. Activer le mode AP (Access Point) dans le système
WIFI_MODE_AP wifi-start

\ 2. Définir le nom du réseau et le mot de passe, puis lancer le Point d'Accès
z" Mon_Reseau_ESP32" z" SuperMotDePasse123" WiFi.softAP
```

Une fois ces lignes exécutées, si vous prenez votre smartphone et que vous scannez les réseaux Wi-Fi disponibles, vous verrez apparaître *"Mon_Reseau_ESP32"*.

WiFi.softAPBroadcastIP (-- n)

Le mot **WiFi.softAPBroadcastIP** permet de récupérer l'**adresse IP de diffusion (Broadcast IP)** du réseau local que l'ESP32 a lui-même créé en mode Point d'Accès (**softAP**).

Pour bien comprendre ce mot, il faut d'abord comprendre le concept de "Broadcast" (diffusion) en réseau.

Dans un réseau informatique, il y a trois façons principales de communiquer :

1. **Unicast** : L'ESP32 parle à un seul appareil précis (ex: le smartphone connecté).
2. **Multicast** : L'ESP32 parle à un groupe d'appareils abonnés.
3. **Broadcast (Diffusion)** : L'ESP32 crie une information à **absolument tout le monde** sur le réseau en même temps. Aucun appareil n'est ignoré.

Par défaut, en mode Point d'Accès, l'ESP32 s'attribue l'adresse IP **192.168.4.1**. L'adresse de broadcast mathématique pour ce type de réseau standard est **192.168.4.255**.

Le mot **WiFi.softAPBroadcastIP** sert à obtenir cette fameuse adresse **192.168.4.255** sans avoir à la deviner ou à la calculer vous-même. Elle retourne un objet de type **IPAddress** (ou une valeur numérique brute selon l'implémentation de votre version d'ESP32forth) directement sur la pile.

C'est particulièrement utile pour :

- **La découverte de services (UDP Beacon)** : Votre ESP32 peut envoyer un paquet UDP "général" toutes les secondes sur cette IP de diffusion. N'importe quelle application smartphone connectée au Wi-Fi de l'ESP32 entendra ce message

et saura instantanément que l'ESP32 est prêt à recevoir des ordres, sans avoir à chercher son adresse IP.

- **Les alertes générales :** Envoyer une notification de coupure ou d'événement critique à tous les clients connectés d'un seul coup.

Généralement, ce mot est utilisé conjointement avec des fonctions réseau (comme les sockets UDP) pour définir la destination d'un paquet.

Voici comment on l'appelle et comment on l'affiche dans la console Forth :

```
\ Activer le point d'accès au préalable...
\ WIFI_MODE_AP wifi-start
\ z" MonESP32" z" 12345678" WiFi.softAP

\ Récupérer et afficher l'IP de broadcast
WiFi.softAPBroadcastIP .ip
```

WiFi.softAPConfig (localip gateway subnet -- success)

Le mot **WiFi.softAPConfig** est le mot qui vous permet de **personnaliser la configuration réseau** du Point d'Accès créé par l'ESP32.

Par défaut, quand vous utilisez **WiFi.softAP**, l'ESP32 choisit des réglages d'usine automatiques (son adresse IP est **192.168.4.1**, le masque est **255.255.255.0**, etc.).

WiFi.softAPConfig permet de modifier ces valeurs de manière précise avant de démarrer le réseau.

Il y a plusieurs raisons cruciales de vouloir configurer manuellement le réseau de l'ESP32 :

- **Éviter les conflits de réseaux :** Si l'un de vos appareils (smartphone ou PC) est déjà connecté à un autre réseau qui utilise la plage **192.168.4.x**, il refusera de communiquer correctement avec l'ESP32. En changeant l'IP en **10.0.0.1** ou **192.168.10.1**, vous éliminez ce risque.
- **Sécurité et Masquage :** Utiliser des plages IP moins communes rend l'infrastructure de votre projet un peu plus discrète.
- **Contrôle de la passerelle (Gateway) :** Vous pouvez définir précisément où les paquets de données doivent être redirigés au sein de votre mini-réseau local.

Le mot **WiFi.softAPConfig** attend généralement **3 adresses IP** distinctes sur la pile Forth :

1. **L'adresse IP locale (local_ip) :** C'est la nouvelle adresse IP que prendra l'ESP32 (par exemple **192.168.10.1**).

2. **La Passerelle (gateway)** : Dans la configuration d'un Point d'Accès isolé, l'ESP32 est sa propre passerelle. On lui donne donc généralement la même adresse IP que son IP locale.
3. **Le Masque de sous-réseau (subnet)** : Il définit la taille du réseau. Généralement, on utilise **255.255.255.0**, ce qui signifie que le réseau pourra accueillir jusqu'à 253 appareils (de **192.168.10.2** à **192.168.10.254**).

En Forth, vous devez d'abord pousser les objets IP (ou leurs représentations chiffrées selon votre version d'ESP32forth) sur la pile, puis appeler la fonction.

Attention à l'ordre : L'appel doit impérativement être fait **avant** de lancer le point d'accès avec **WiFi.softAP**.

Voici l'exemple typique de mise en œuvre :

```
\ 1. Préparer le mode Point d'Accès
WIFI_MODE_AP wifi-start

\ 2. Définir les adresses IP (Syntaxe théorique selon l'implémentation de vos
mots IP)
\ Ici, on configure l'ESP32 sur l'IP 192.168.10.1
192.168.10.1  ip-addr      \ Pousse l'IP locale
192.168.10.1  ip-addr      \ Pousse la passerelle (elle-même)
255.255.255.0 ip-addr      \ Pousse le masque de sous-réseau

\ 3. Appliquer la configuration
WiFi.softAPConfig

\ 4. Démarrer le Wi-Fi avec le nom et le mot de passe
z" Mon_Reseau_Perso" z" 12345678" WiFi.softAP
```

Une fois ce code exécuté, votre ESP32 n'écouterait plus sur **192.168.4.1**, mais répondrait désormais à l'adresse **192.168.10.1**. De même, le mot **WiFi.softAPBroadcastIP** retournerait automatiquement **192.168.10.255**.

WiFi.softAPdisconnect (fl – success)

Le mot **WiFi.softAPdisconnect** permet de **couper et de désactiver le Point d'Accès (Access Point)** actuellement géré par l'ESP32.

C'est l'action inverse de **WiFi.softAP**. Lorsque ce mot est exécuté, l'ESP32 arrête de diffuser son réseau Wi-Fi (le SSID disparaît des scans) et déconnecte instantanément tous les appareils (smartphones, PC) qui y étaient connectés.

Il y a trois scénarios principaux où ce mot est indispensable :

- **Économie d'énergie** : Maintenir un point d'accès Wi-Fi actif consomme beaucoup d'électricité. Si l'ESP32 n'a besoin du réseau que pendant 5 minutes (le temps qu'un

utilisateur configure l'appareil), il est crucial de fermer le point d'accès juste après pour préserver la batterie ou réduire la chauffe du composant.

- **Transition vers le mode Station** : Souvent, l'ESP32 démarre en mode Point d'Accès pour que l'utilisateur saisisse les identifiants de sa box internet domestique. Une fois reçus, l'ESP32 ferme son propre point d'accès avec **WiFi.softAPdisconnect** avant de basculer en mode Station pour se connecter à la box.
- **Sécurité** : Laisser un point d'accès ouvert ou actif en permanence sans raison expose l'ESP32 à des tentatives de connexion indésirables ou à des attaques par déni de service (DoS).

Grâce à cette définition en C, vous pouvez tester si la déconnexion a réussi avant de passer à l'étape suivante de votre programme :

```
\ On demande l'arrêt du softAP ET l'extinction de la radio (true)
true WiFi.softAPdisconnect ( -- success )
```

Si vous n'avez pas besoin de vérifier le résultat (**success**), vous devez simplement ajouter le mot **drop** juste après pour nettoyer la pile et éviter qu'elle ne déborde.

WiFi.softAPIP

WiFi.softAPNetworkID

WiFi.softAPgetStationNum

WiFi.status

WiFi.linkLocalIPv6

Non défini pour le moment,

WiFi.globalIPv6

Non défini pour le moment,

Index lexical

WIFI_MODE_AP.....	7	WiFi.localIP.....	12
WIFI_MODE_APSTA.....	8	WiFi.macAddress.....	12
WIFI_MODE_NULL.....	9	WiFi.mode.....	13
WIFI_MODE_STA.....	10	WiFi.scanNetworks.....	16
WiFi.begin.....	10	WiFi.setTxPower.....	15
WiFi.config.....	10	WiFi.softAP.....	17
WiFi.disconnect.....	11	WiFi.softAPBroadcastIP.....	18
WiFi.enableIPv6.....	12	WiFi.softAPConfig.....	19
WiFi.getTxPower.....	14	WiFi.softAPdisconnect.....	20

